Technical report — Sigillum

A sovereignty seal for AI tools, from A to E, with proof behind every answer.

- **Track:** Track 2B — Own Project (Apertus Adoption)
- **Event:** Hack Apertus — Online, 1–16 October 2026
- **Team:** Sigillum — Oren
- **Demo:** `docs/video/sigillum-demo.mp4` (1 min 43 s; English narration in Oren's voice, English subtitles)

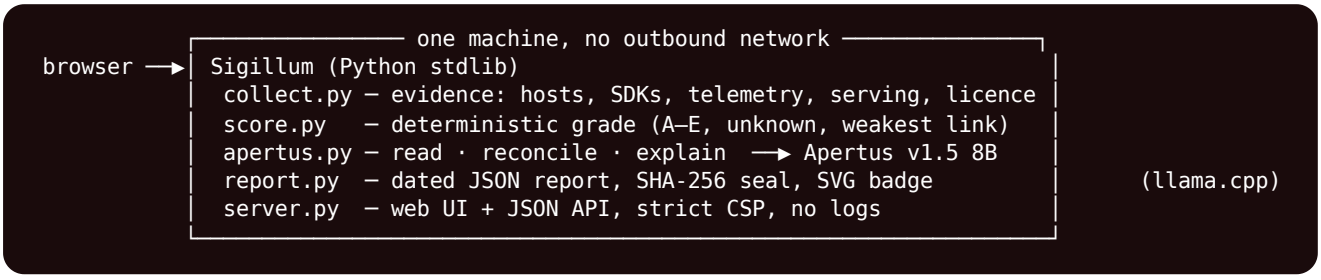
1. Summary

"Sovereign" is the most used word in European AI and the least verified. A tool can run on an open model and still send its users' questions to a foreign API, ship analytics, or stop working when the connection drops. Sigillum turns that vague promise into a measurement. It asks any AI project five questions — where does the model live, where does the data go, what happens without internet, who else is in the loop, can it be verified — finds a verifiable trace (file and line) for each answer, and summarises the result as one letter from A (fully sovereign) to E (dependent on foreign services) with five pictograms, like the energy label or the Nutri-Score.

The grade is deterministic and rests on three published rules: the **proof rule** (a claim without a trace never earns more than C; missing information is shown as *unknown* and caps the grade at C), the **weakest link** (the final grade is the worst answer) and a **dated, tamper-evident seal** (commit, method version, SHA-256 fingerprint, 90-day expiry). **Apertus does what fixed rules cannot**: it extracts what a project claims about itself, sets each claim against the traces and names the contradictions, and writes the verdict in German, French, Italian and Romansh. Every Apertus answer is checked against the evidence before use, and rejections are counted in the report.

Headline results: on 5 reference projects, the seal matches the expected grade in 5/5 cases, inside a container with networking disabled. Applied to the 26 public Hugging Face Spaces that use Apertus (aggregated, anonymous), no project reaches A or B; 42 % call the model through a foreign service and 73 % ship telemetry on by default. Sigillum examined itself first: D on its first run (no licence file in the project folder), A after the fix, with its limits published.

2. Architecture



Component	Role	Runs
`collect.py`	Walks the project (skips `node_modules`, `.git` …), separates runtime files, build files and docs; records evidence with file, line, snippet, host, operator, jurisdiction	in-process
`registry.py`	Public, hand-curated registry of 123 destinations: operator, jurisdiction (LOCAL, CH, EU, US, CN, CA, GLOBAL, UNKNOWN), category (closed LLM, open-model inference, analytics, telemetry, CDN, fonts, tunnel, storage, auth, model hub, docs). `.ch` domains are marked "Swiss TLD heuristic"	data
`score.py`	Five questions → level A–E or unknown; proof type (reproducible / trace / declaration / none); fixes; rule-based gaps	in-process
`apertus.py`	OpenAI-compatible client (urllib) with JSON-schema structured output, deterministic decoding (temperature 0, fixed seed), response cache, and validation of every answer	calls `LLM_BASE_URL`
`report.py`	Report assembly, fingerprint, expiry, badge	in-process
`server.py`	Static UI + API (`/api/reports`, `/api/audit`, `/api/badge/<id>.svg` …). Audit targets restricted to bundled samples, a mounted `/audit` folder, or public https Git URLs	port 8080
`community.py`	Aggregated, anonymous audit of a folder of public projects; only proportions are written	in-process

The web interface follows America.gov's design system (display serif at −0.05 em tracking, tight sans body, 48 px hero shell, pill controls, soft elevation, oversized footer wordmark) in Swiss colours: flag red RGB 255/0/0 (Pantone 485 C) for the cross mark, #DA291C for interactive elements (white text contrast 4.9:1). Fonts (Source Serif 4, Inter — OFL) and icons are bundled; the page loads nothing from third parties.

Target architecture (mandatory)

(b) Air-gapped, and therefore also (a) on-premise. Sigillum has zero runtime dependencies (Python standard library), makes no outbound request except to the configured Apertus endpoint, and serves Apertus locally with llama.cpp in the same `docker compose`. `make offline-test` runs the whole test suite and every sample audit in a container with `network_mode: none` (log: `docs/offline-test.log`). It can equally point `LLM_BASE_URL` at a sovereign Swiss endpoint (c); each report shows the endpoint's jurisdiction.

External dependencies — **build time only:** `python:3.12-slim` base image and `git` (Debian packages), `ghcr.io/ggml-org/llama.cpp:server` image, Apertus v1.5 8B GGUF weights from Hugging Face (`make model`, ~5 GB). **Runtime:** none. Examining a remote Git URL needs network access to that host; air-gapped deployments mount folders under `/audit` instead.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B`, text-only conversion quantised to Q4_K_M GGUF (`Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`), served by llama.cpp (`llama-server --jinja`, context 6–8k). Any Apertus endpoint works through `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_NAME` (e.g. the 70B on CSCS).
- **How it is used:** inference with structured output (JSON schema), three tasks:
  1. **Read** — extract the sentences in which the documentation claims something about model location, data flows, offline use, third parties or verifiability. *Check:* the quote must exist verbatim (normalised) in the documentation.
  2. **Reconcile** — one verdict per claim: is it contradicted by a trace? Apertus only sees traces that can contradict (foreign flows, telemetry, downloads, opaque files). *Check:* the claim ID must exist and every cited evidence ID must be one of those traces; anything else is rejected, counted and logged.
  3. **Explain** — one plain sentence per question in fr, de, it, rm, en, for an elderly reader or a pupil. *Check:* exactly one line per question, at most 45 words, and a function-word language identifier must recognise the target language; failed lines fall back to reviewed templates, marked as such.

- **Where it runs:** local weights on the same machine (4 CPU cores, no GPU): ~38 tokens/s prompt, ~4.5 tokens/s generation; about 5–6 minutes per project for 7 calls. Responses are cached by prompt hash, so re-examining an unchanged project is instant and reproducible.
- **What Apertus never does:** set or change a grade. The grade is computed before Apertus is called and is part of the fingerprint.

Other models: none. No other LLM was used to build, evaluate or judge Sigillum's outputs.

4. Data

- **Reference projects** (`data/samples/`, 5 projects): written by the Sigillum team for this hackathon, fictional and labelled as such, Apache-2.0. They reproduce the scenes of the brief: a municipal assistant served locally (A), a vendor shipping a compiled bundle (C, with *unknown* on question 4), a closed-model SaaS (E), and a school assistant before and after fixes (D → B).
- **Community audit** (`data/community/aggregate.json`): the 26 public Hugging Face Spaces returned by the Hub search "apertus" on 16 Oct 2026, shallow-cloned without LFS files, analysed statically, then deleted. Only aggregate proportions are stored; folder names were anonymised before analysis and the aggregation code cannot output names. No request was made to the running applications and no data about users was collected. Hack Apertus submissions were not examined: they become public after 24 October, and will only be named with their teams' consent.
- No personal data is processed. Reports contain file paths and code snippets of the examined project only.

5. Evaluation

Correctness on reference projects (deterministic part; `make test`, 18 tests):

Project	Expected	Sigillum	Key evidence
municipal-assistant	A	A	llama.cpp service, internal network, offline test, pinned deps
town-hall-chat	C, Q4 unknown	C, Q4 unknown	configurable endpoint only; minified vendor bundle cannot be audited
civicgpt	E	E	OpenAI SDK default endpoint, Mixpanel, Vercel Analytics, Google Fonts, no licence
school-assistant v1	D	D	Streamlit telemetry on by default, CDN script, weights downloaded at start-up
school-assistant v2	B	B	telemetry off, Apertus on llama.cpp, no offline test yet
Sigillum itself (1st run)	—	D	no licence file in the project folder (real finding)
Sigillum itself (after fix)	A	A	stdlib only, local Apertus, offline test in Docker

Apertus checks (5 reference projects + Sigillum): claims extracted are verbatim; the contradiction "*Everything runs locally: no data leaves the school.*" vs. Streamlit telemetry (US) and a CDN script was found by Apertus with valid evidence IDs, matching the rule-based gap. During development, the checks caught Apertus writing French or German explanations in English, and copying French into Romansh; those lines were rejected and replaced by templates. The final count of calls and rejections is printed in each report (`apertus.calls`, `answers_rejected_by_checks`).

Apertus hallucinating contradictions — on the A-graded reference project, the first reconciliation run produced four "gaps" whose cited evidence was positive (the local llama.cpp server, the open model name). We now pass Apertus only traces that can contradict a claim (foreign flows, telemetry, runtime downloads, opaque files, closed models) and reject any gap citing other evidence (test

`test_gap_citing_positive_evidence_is_impossible`). Rejected answers are logged in each report (`apertus.rejections`) as feedback for the Apertus team.

Rigour against false positives — the community audit was first run with a naive extractor and inspected manually: hyperlinks to documentation (`docs.streamlit.io`, `vuejs.org`, citation links) were counted as data flows, and licences declared in Space metadata (`license:` front matter) were missed. Both were fixed (hyperlink heuristic, README metadata licence) and covered by tests before publishing the aggregate. Grades of the 26 Spaces after the fixes: A 0, B 0, C 4, D 11, E 11.

Cost — deterministic examination: 1.9 s for 26 projects (~70 ms each) on a CPU. With Apertus on 4 CPU cores: ~5–6 minutes per project, CHF 0 marginal cost, nothing leaves the machine.

6. Limitations

- Static analysis only: behaviour that appears only at runtime (URLs built dynamically, plugins, remote configuration) can be missed. A runtime probe (proxy recording outbound connections during the project's own tests) is the next step.
- The destination registry is hand-curated and incomplete; unknown hosts are reported as *unknown*, never as safe. `.ch` is a heuristic, not proof of Swiss operation.
- Hyperlink exclusion is heuristic; a script that fetches a URL written as a link could be missed.
- Build-time downloads are listed but do not lower the runtime grade.
- `.sigillumignore` exclusions are self-declared; they are published in every report. Sigillum's own exclusions (its detection tables and texts, tests, sample data) are visible on its seal.
- Apertus 8B Q4 on CPU is slow and conservative: its plain-language lines stay close to the reviewed templates, it sometimes answers in the wrong language (caught by the checks), and Romansh should be proof-read by a native speaker. The 70B model on a GPU endpoint would likely produce richer explanations.
- Sigillum is a transparency instrument, not a legal label or an official certification.

7. Reproducibility

- Hardware used: 4 vCPU, 16 GB RAM, no GPU (cloud container). Runtime: Python 3.12, llama.cpp `llama-server` (build of 16 Oct 2026), Apertus v1.5 8B Q4_K_M.
- Decoding: temperature 0, seed 7, JSON schema; responses cached in `data/cache/` by SHA-256 of the request, so `make samples` reproduces the bundled reports exactly.
- `make test` (unit tests), `make offline-test` (tests + all sample audits with `--network none`), `make run` (web UI), `python3 -m sigillum verify data/reports/<id>.json` (seal fingerprint).
- The community aggregate is rebuilt with `make community` (needs internet once to clone the public Spaces).

8. Next steps

- Runtime probe: record outbound connections while the examined project's own tests run, to turn "inferred" answers into "reproducible" ones.
- A self-service flow for teams of the on-site hacks: request a seal, read the report, accept or contest before anything is published.
- Run the explanations on Apertus 70B (CSCS) and have native speakers review the Romansh, Swiss German and Italian templates.
- Grow the registry with Swiss providers and let organisations contribute entries by pull request.

- Offer the seal to the organisers as an evaluation aid for the on-site hacks and the Grand Finals in May 2027.

License

Code: Apache-2.0. This report and the method: Creative Commons Attribution 4.0 (CC-BY-4.0). All Hack Apertus projects are open-sourced.

References

- Apertus v1.5 models — huggingface.co/swiss-ai/Apertus-v1.5-8B, huggingface.co/swiss-ai/Apertus-v1.5-70B
- llama.cpp — github.com/ggml-org/llama.cpp
- Swiss Federal Act on the Protection of the Swiss Coat of Arms and Other Public Signs (232.21), flag colour specification
- America.gov design (National Design Studio) — america.gov
- Streamlit configuration `browser.gatherUsageStats`; Gradio `GRADIO_ANALYTICS_ENABLED`; Chroma `anonymized_telemetry`; vLLM `VLLM_NO_USAGE_STATS` — respective documentation